Week-7 | Summary

Karthik Thiagarajan

1. Common 2
    1.1. Notation 2
    1.2. Labeled Dataset 2
    1.3. Data-matrix 3
    1.4. Label Vector 3
    1.5. Data-point 3
    1.6. Label set 3
2. Zero-One Error 4
3. Linear Classifier 4
4. KNN 6
5. Decision Trees 9
    5.1. Binary tree 9
    5.2. Entropy: Node impurity 10
    5.3. Decision Stump 11
    5.4. Growing a Tree 12
6. References 17

1. Common

1.1. Notation

Scalars:

$$x_1, x_2, y_1, y_2, z_2, z_2, a, b, \alpha, \beta$$

Column vector:

$$\mathbf{x} \in \mathbb{R}^d$$

$$\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_d \end{bmatrix}$$

Row vector:

$$\mathbf{x} \in \mathbb{R}^d$$

$$\mathbf{x}^T = \begin{bmatrix} x_1 & \cdots & x_d \end{bmatrix}$$

Matrix:

$$\mathbf{X} \in \mathbb{R}^{d \times n}$$

1.2. Labeled Dataset

$$D = \{(\mathbf{x}_1, y_1), \ \cdots, (\mathbf{x}_n, y_n)\}$$

1.3. Data-matrix

$$\mathbf{X} \in \mathbb{R}^{d \times n}$$

- $d \rightarrow$ number of features
- $n \rightarrow$ number of data-points

$$\mathbf{X} = \begin{bmatrix} | & & | \\ \mathbf{x}_1 & \cdots & \mathbf{x}_n \\ | & & | \end{bmatrix}$$

1.4. Label Vector

$$\mathbf{y} \in \{0, 1\}^n$$

or

$$\mathbf{y} \in \{-1, 1\}^n$$

where

$$\mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$$

1.5. Data-point

$$\mathbf{x}_i \in \mathbb{R}^d$$

1.6. Label set

For binary classification, the label set could be $\{0, 1\}$ or $\{1, -1\}$.

2. Zero-One Error

If $f : \mathbb{R}^d \to \{1, 0\}$:

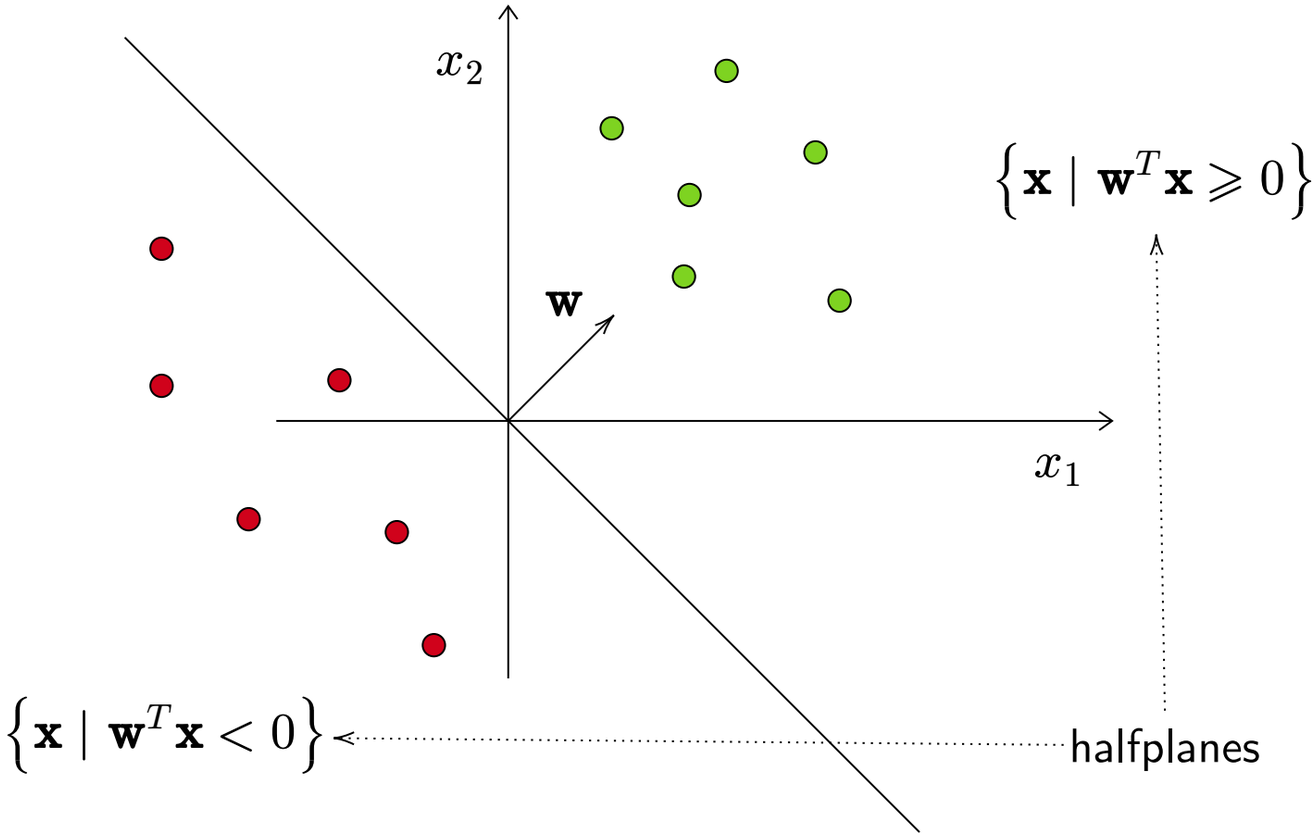
$$L(D, f) = \frac{1}{n} \sum_{i=1}^{n} \mathbb{I}[y_i \neq f(\mathbf{x}_i)]$$

This is the average number of errors in prediction by the classifier.

3. Linear Classifier

$$f : \mathbb{R}^d \to \{1, -1\}$$

$$f(\mathbf{x}) = \text{sign}\left(\mathbf{w}^T \mathbf{x}\right) = \begin{cases} 1, & \mathbf{w}^T \mathbf{x} \geqslant 0 \\ -1, & \mathbf{w}^T \mathbf{x} < 0 \end{cases}$$

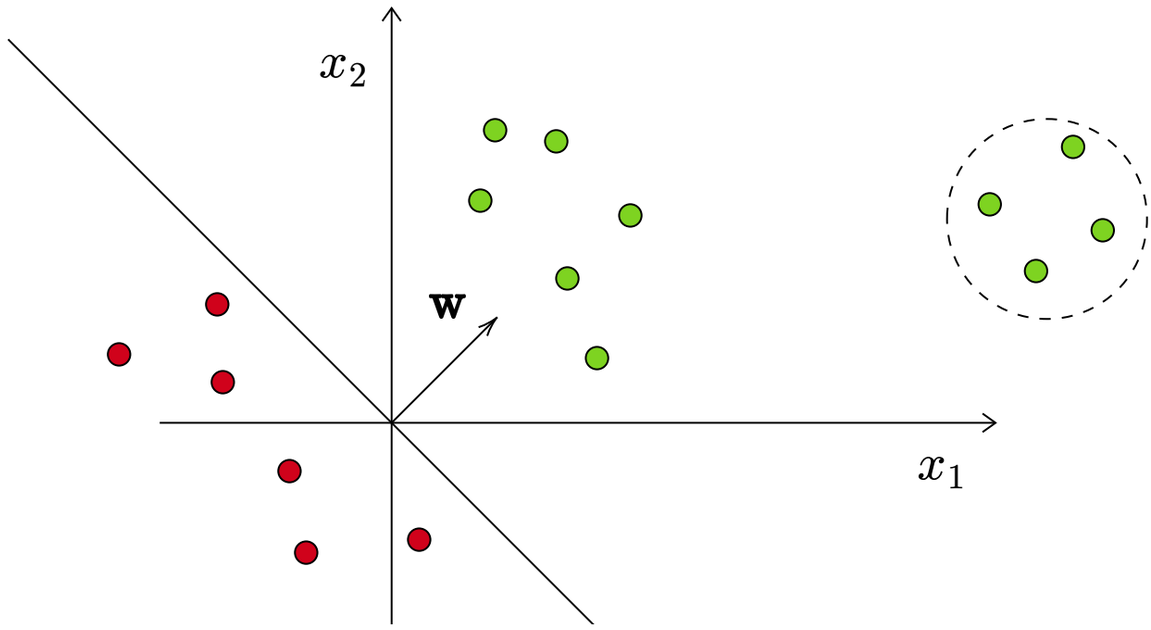


The optimization problem given below is NP hard.

$$\min_{\mathbf{w}} \ \frac{1}{n}\sum_{i=1}^{n} \mathbb{I}\Big[y_i \neq \text{sign}\big(\mathbf{w}^T\mathbf{x}_i\big)\Big]$$

Using the SSE for classification is not a good idea. The SSE is sensitive to outliers.

$$\min_{\mathbf{w}} \ \frac{1}{n}\sum_{i=1}^{n} \big(\mathbf{w}^T\mathbf{x}_i - y_i\big)^2$$



Besides, modeling classification as a regression problem has another issue. There is no natural ordering among the labels in a classification problem. For example, if the labels are "cat", "dog", "mouse", all permutations of the labels $\{1, 2, 3\}$ are equally valid. A regression based approach would however take into account the ordering between $\{1, 2, 3\}$. There is a definite sense in which $3 > 2 > 1$ and $3 - 2 = 2 - 1 = 1$. However, no such order exists among the labels "cat", "dog" and "mouse".

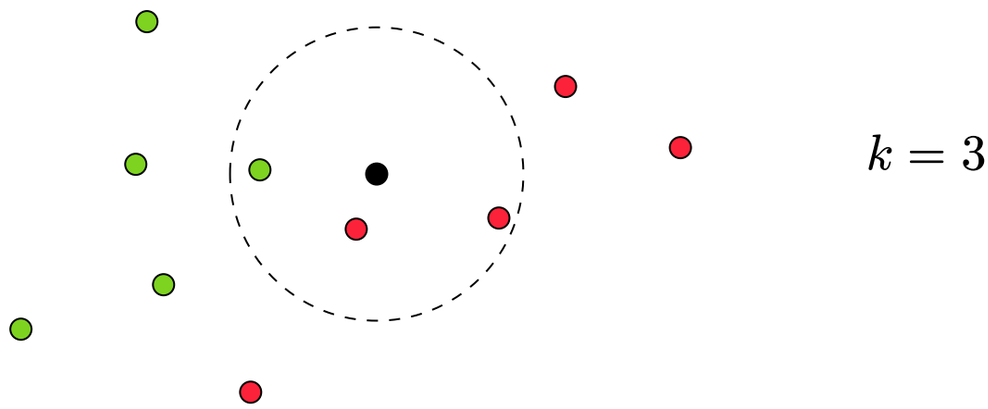
4. KNN

Prediction for a given test-point:

- Find $d(x_{\text{test}}, x_i)$ for all i
- Sort distances in ascending order
- Find the labels of the first k points
- Return the most frequently occurring label

Usually an odd number is chosen for k to avoid ties.
Hyperparameters:

- k
- distance metric
  - L_2
  - L_1



<u>Dataset</u>

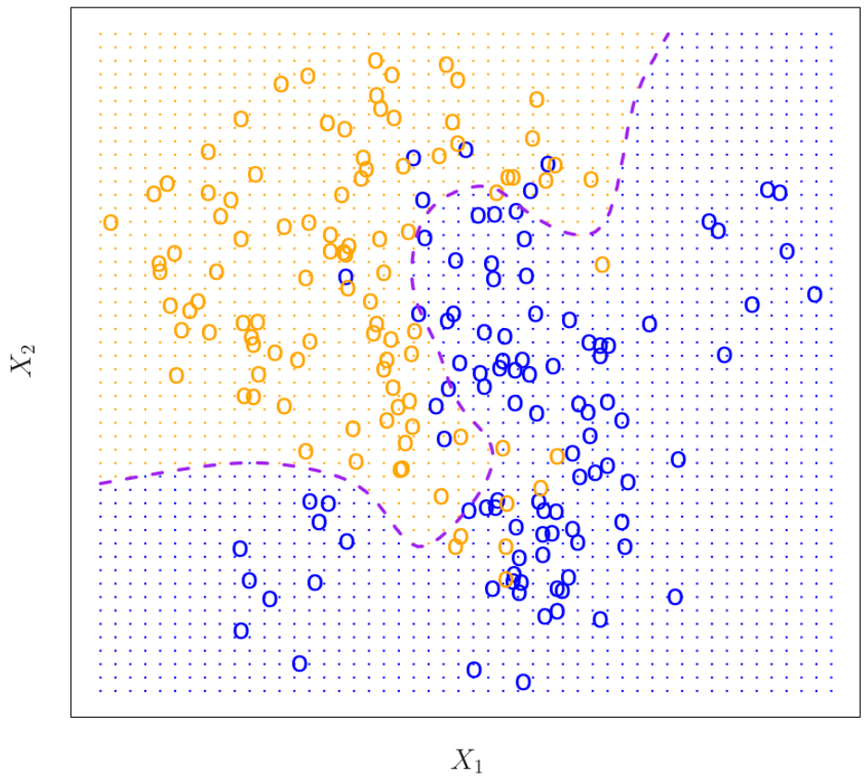


Figure 1: Source: Pg 36, ISLP

<u>Effect of k on decision boundary</u>

As k increases, the model becomes less flexible.

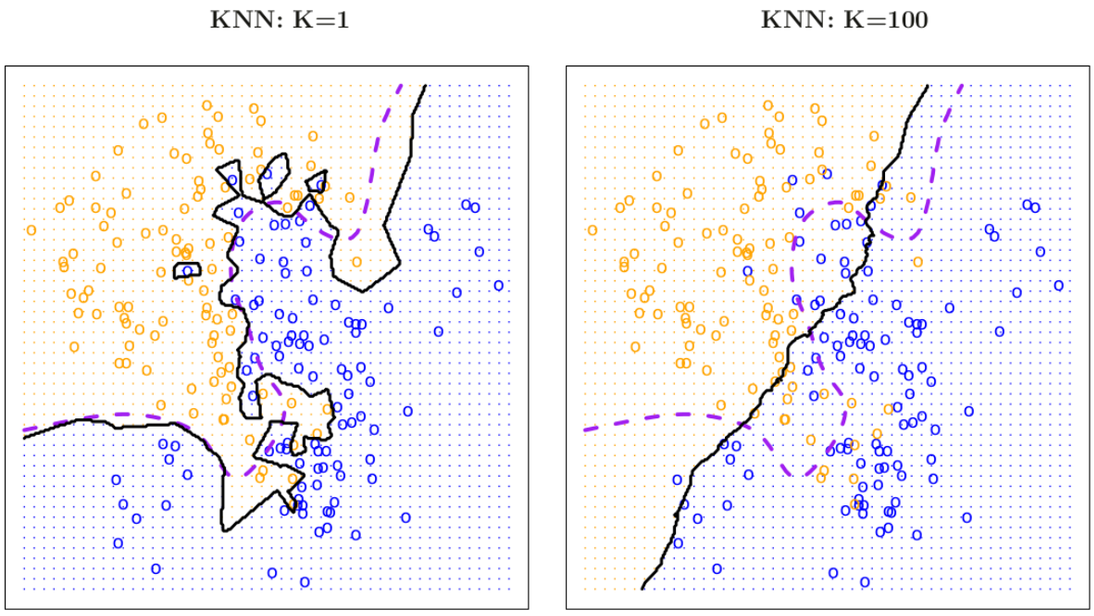


Figure 2: Source: Pg 39, ISLP

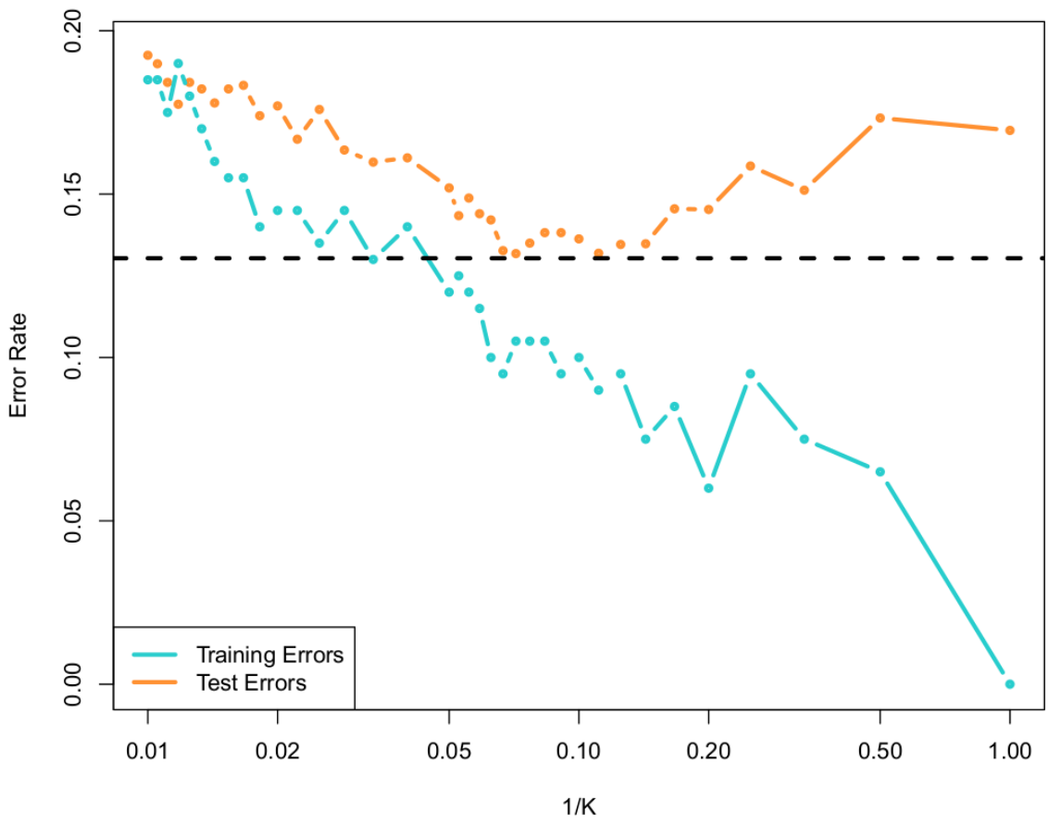


Figure 3: Source: Pg 39, ISLP

<u>Advantages</u>

- Very easy to implement
- Interpretable
  - Increases trust in the model

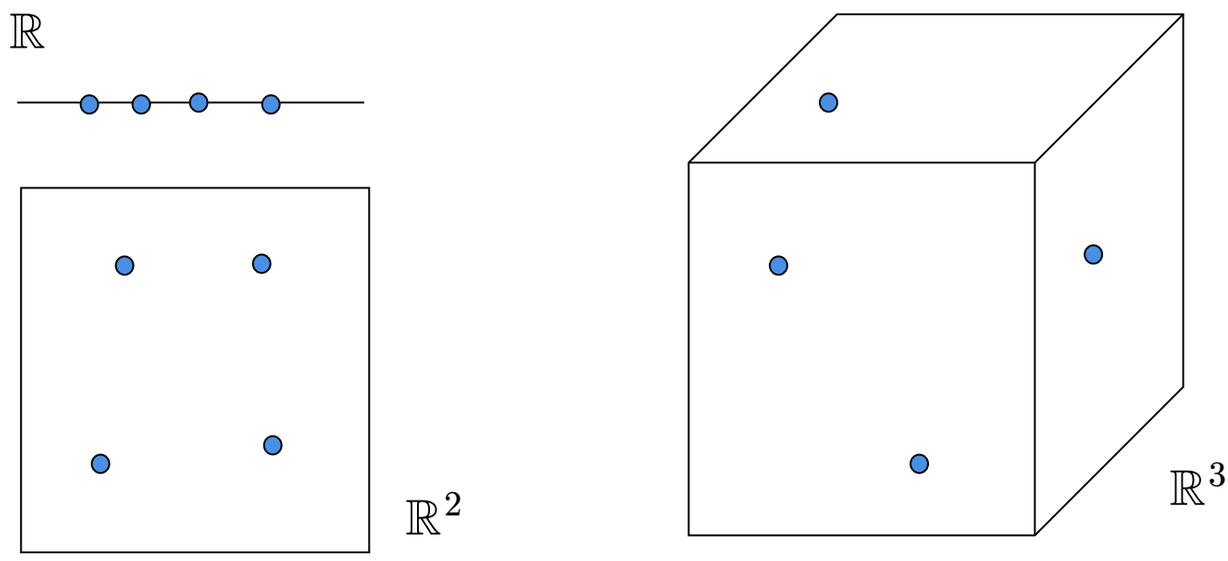
- Easy to explain to non-experts

<u>Disadvantages</u>

- Computationally expensive
  - n points in training data
  - n distances have to be computed for each prediction
  - n distances have to then be sorted
- No model is learnt
- Training data has to be stored even during prediction
- Curse of dimensionality

<u>Curse of Dimensionality</u>

As dimensions increase, neighborhood information becomes less reliable.



5. Decision Trees

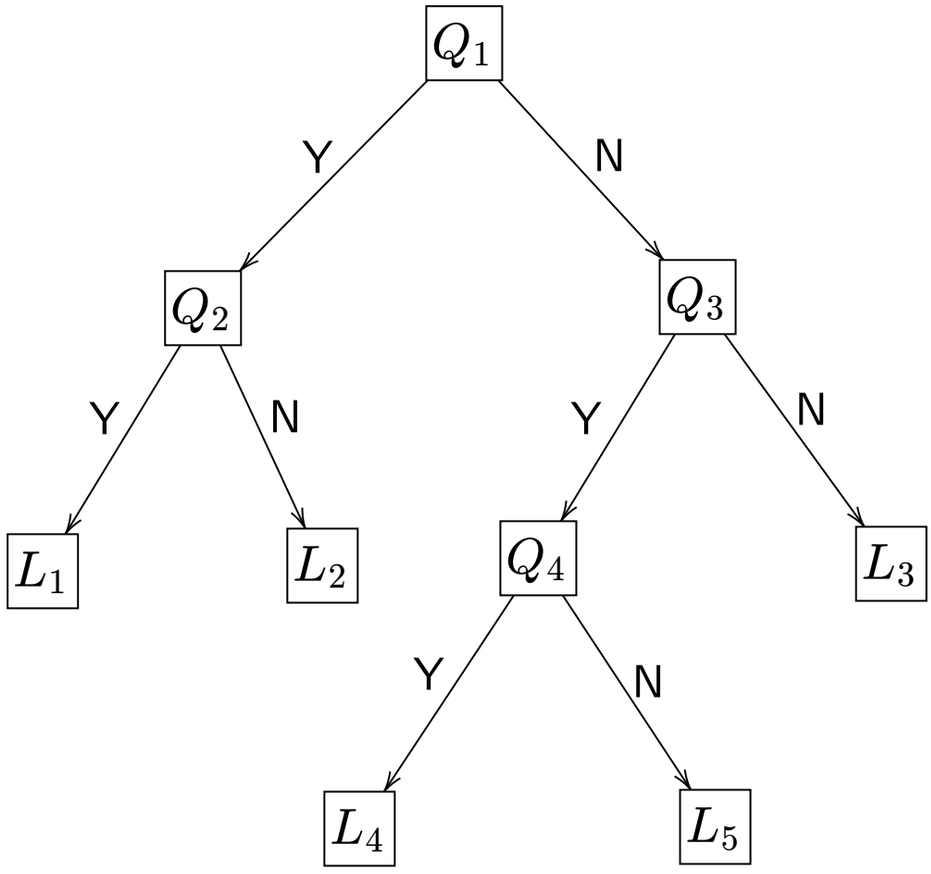
5.1. Binary tree

- Q_i : feature $<$ value
- Depth $= 3$
- Nodes
  - Root node: Q_1
  - Internal nodes: Q_2, Q_3, Q_4

– Leaves: L_1, L_2, L_3, L_4, L_5

The tree is the model. Once the tree has been grown from data, the data can be thrown away.

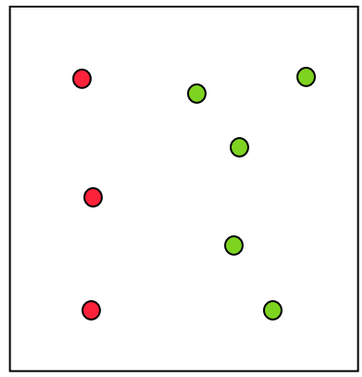
Depth: The number of edges on the longest path from the root to a leaf.



5.2. Entropy: Node impurity

- $n_P \rightarrow$ number of positive data-points
- $n_N \rightarrow$ number of negative data-points

$$n = n_P + n_N$$

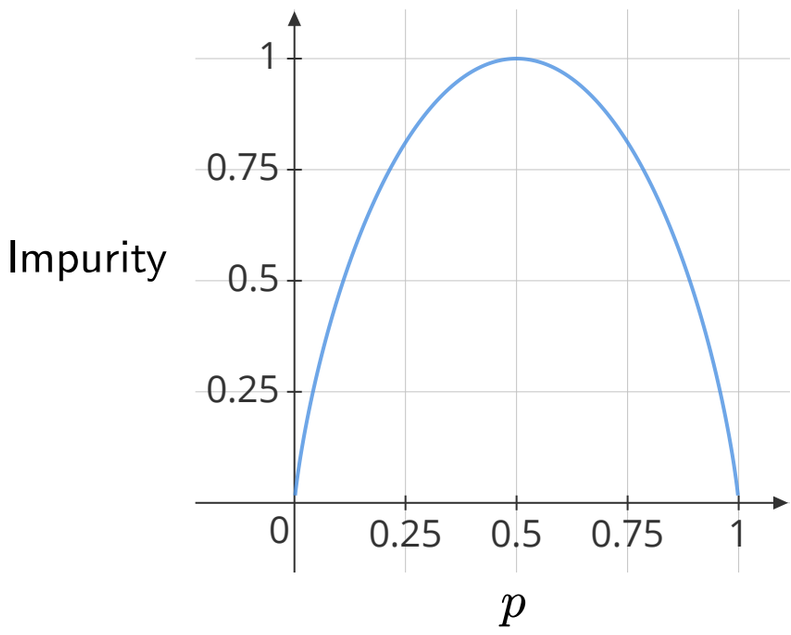


$p \rightarrow$ proportion of positive (negative) data-points in a node

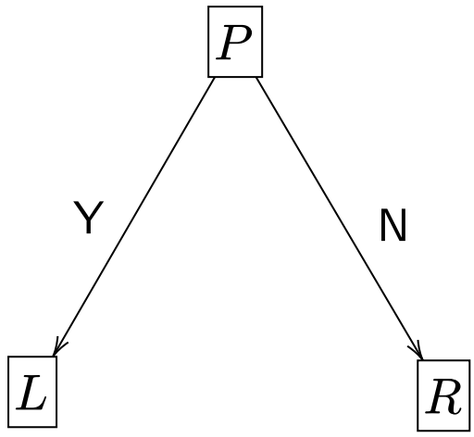
$$p = \frac{n_P}{n}$$

The entropy of a node is a measure if the node's impurity:

$$H(p) = -p\log_2 p - (1-p)\log_2(1-p)$$



5.3. Decision Stump



- D: dataset at the parent
- $x_f < s$: question
- D_L and D_R: partitions
- p_P, p_L, p_R: proportions at P, L, R
- γ: proportions of points in L
- E_P: entropy of P
- E_L: entropy of L
- E_R: entropy of R
- IG: information gain

$$n = n_L + n_R$$

$$\gamma = \frac{n_L}{n}$$

$$E = -p \log_2 p - (1-p) \log_2 (1-p)$$

$$IG = E_P - [\gamma E_L + (1-\gamma) E_R]$$

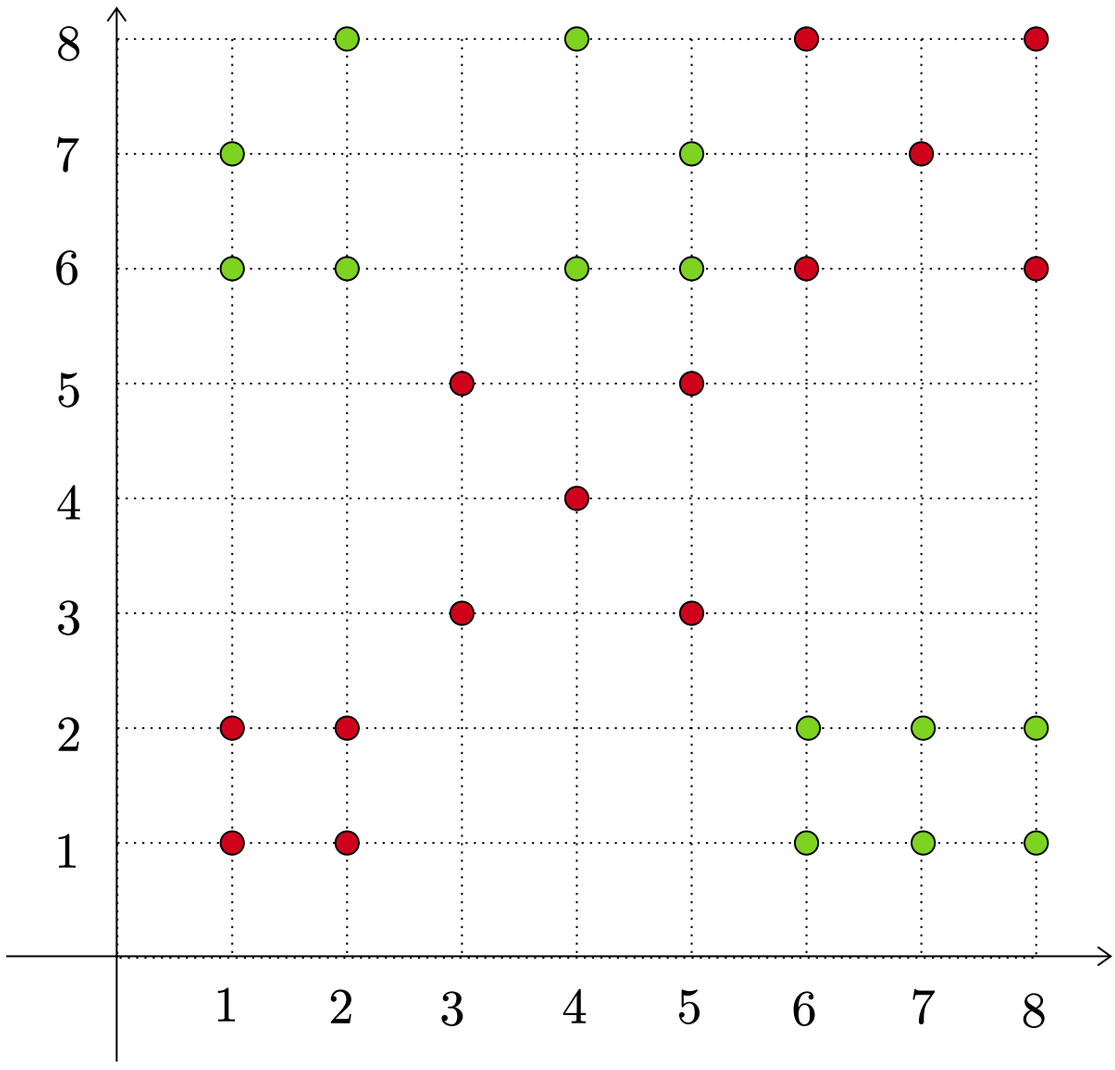
5.4. Growing a Tree

Growing a tree is a recursive and greedy algorithm:

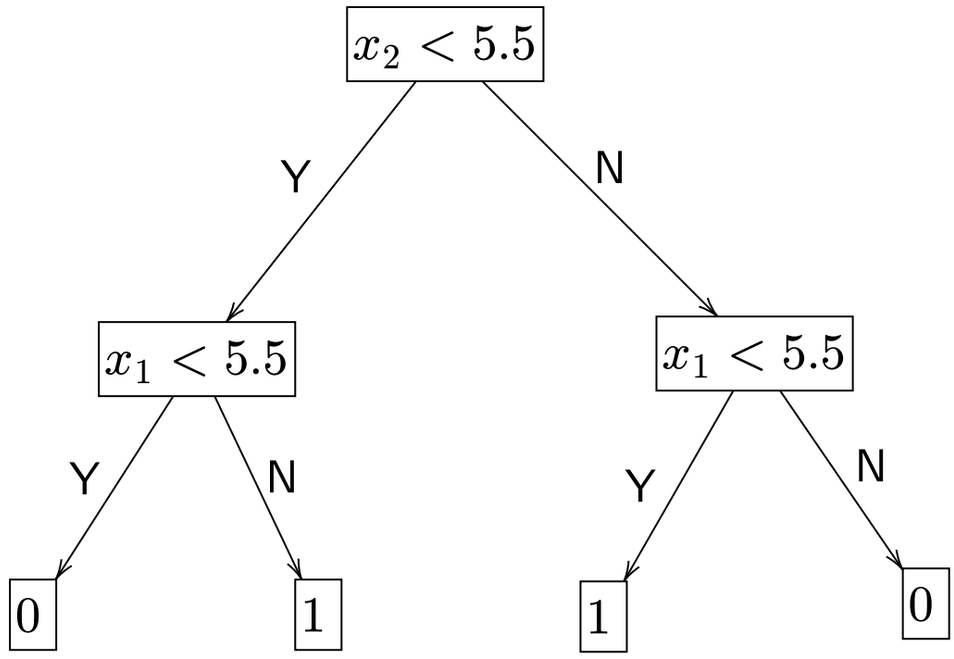
- At each node, choose the (feature, value) pair that gives the greatest reduction in entropy or the maximum gain in information.
- Choice of values for a given feature
  - Sort all the values for this feature in the training dataset
    * Use this set of values (OR)
    * Use the mid-point between consecutive feature values in this set.
- Stop growing the tree when the stopping criterion is met

  - Default stopping criterion is when a node is completely pure
  - Other stopping criteria are discussed in the end.
- Assign labels to leaf nodes
  - Label is most frequently occurring label

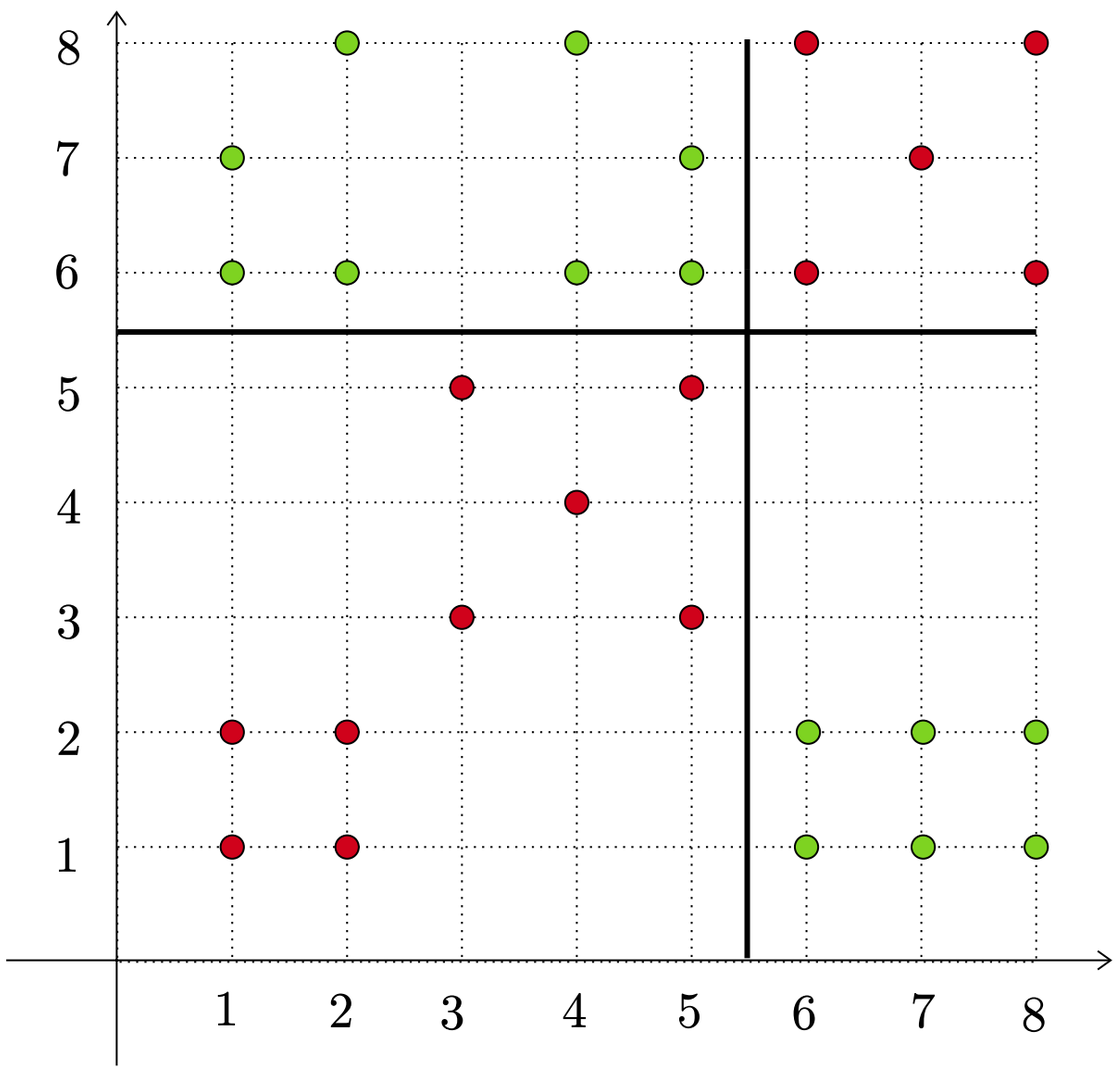
<u>Sample dataset</u>



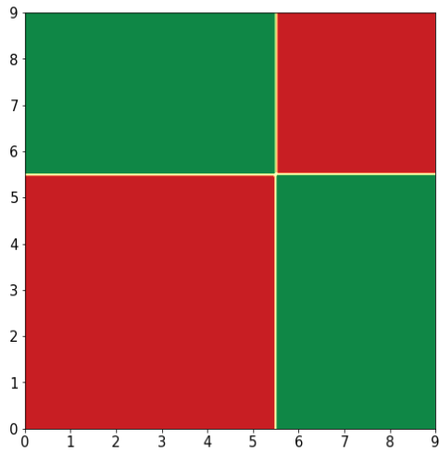
<u>Tree</u>



Decision boundary



Decision Regions



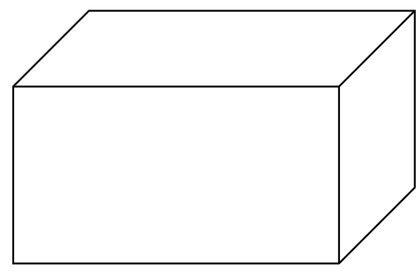
- 4 regions
- 4 leaves

In general, boundaries of the regions are parallel to the axes:

$\mathbb{R}$

$\mathbb{R}^2$

$\mathbb{R}^3$



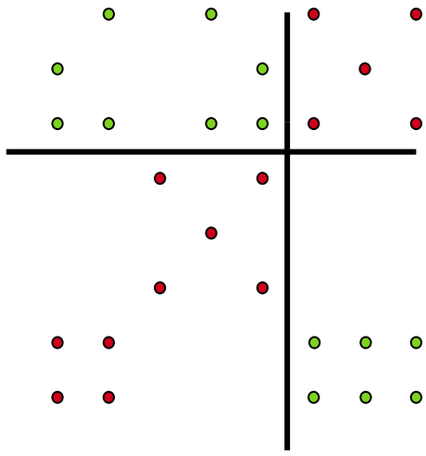
$\mathbb{R}^d$

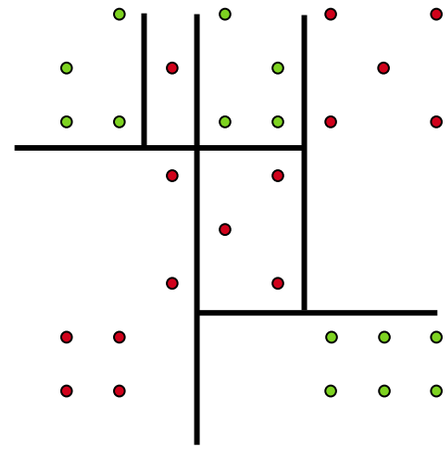
hyper-rectangles

<u>Prediction</u>

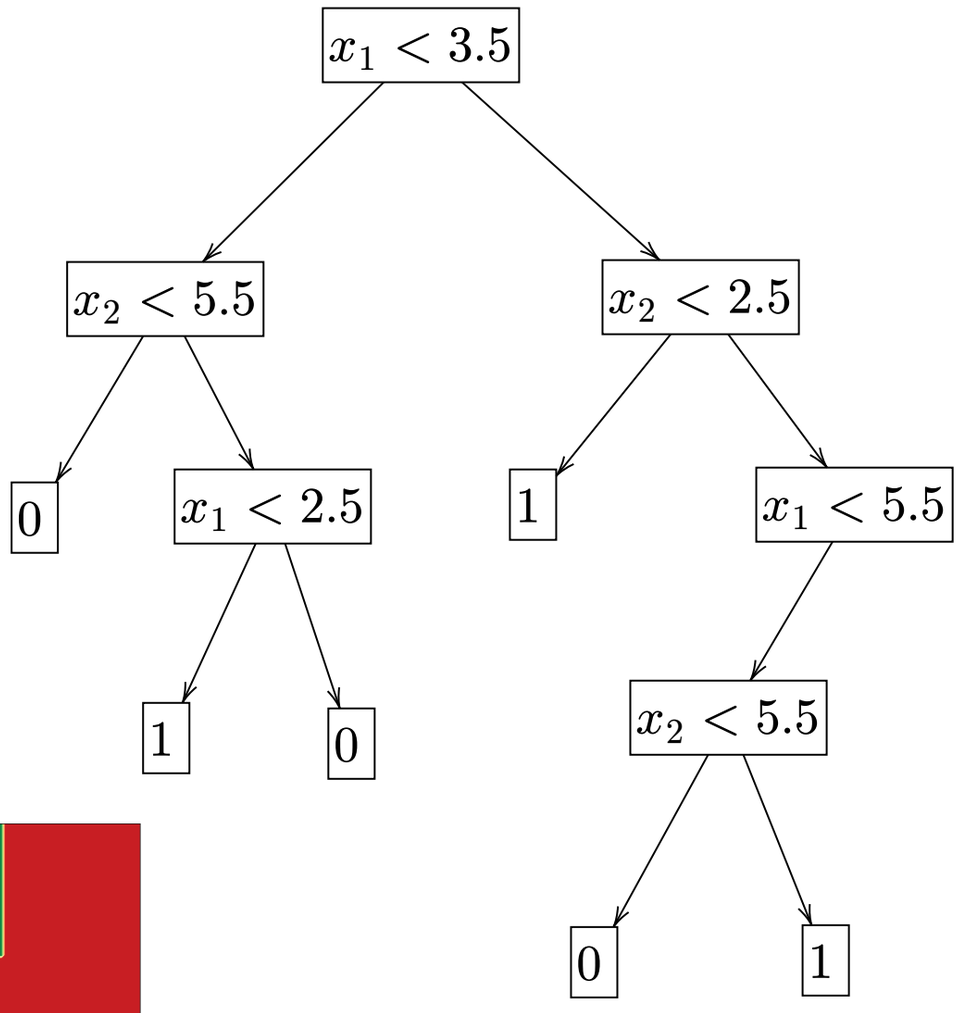
Traverse the tree. When a leaf node is reached, output the label of the leaf node.

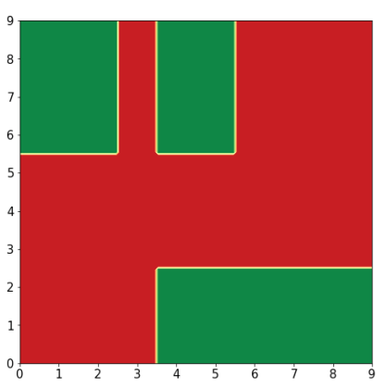
<u>Overfitting</u>





Deep trees have high variance.





- 7 regions
- 7 leaves

<u>Mitigation</u>

- Minimum samples at leaf node
- Maximum depth

- Minimum decrease in impurity

<u>Advantages</u>

- Shallow trees are interpretable
- Easy to implement

<u>Disadvantages</u>

- Simple model with low predictive power
- Deep trees have high variance
  - small changes to the training data result in big changes in the resulting tree

6. References

- Prof Arun Rajkumar's lectures in MLT
- ISLP: Introduction to Statistical Learning using Python